

Table of contents

1	The Main Loop	2
1.1	Animation	2
1.2	System Architecture	2
1.3	The Game Loop	3
1.4	Conclusion	3

Chapter 1

The Main Loop

We have the canvas, memory, mathematics, and physics. Now, we bring them to life.

1.1 Animation

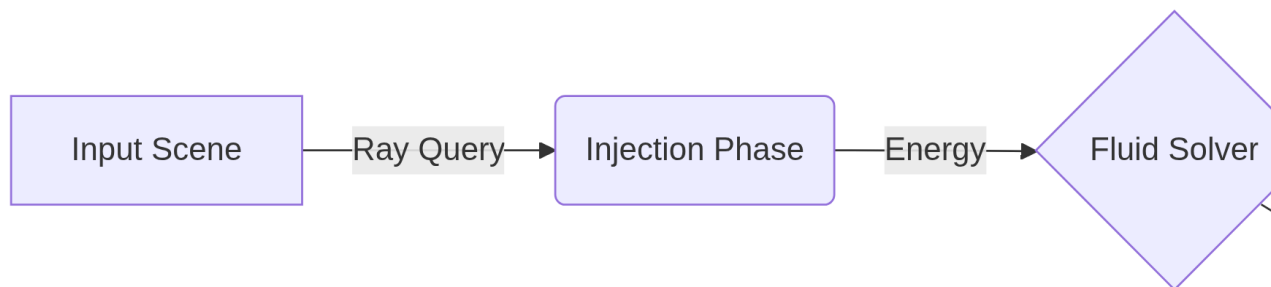
Animation is an illusion. By updating the screen 60 times per second (60 Hz), we trick the eye into seeing motion.

JavaScript provides `requestAnimationFrame`. This instructs the browser: “When ready to paint the screen, call my function first.”

Why not
`setInterval`?
Because
`requestAnimationFrame`
pauses when you
switch tabs,
saving battery!

1.2 System Architecture

Before viewing the code, let us observe the “Big Picture”. The engine runs in three phases:



1.3 The Game Loop

Our `mainSimulationLoop` acts as the orchestra’s conductor. It runs continuously, coordinating all discussed components.

```
function mainSimulationLoop() {  
  // 1. Handle Input (Keyboard/Mouse)  
  if (handleInput()) {  
    // If the user moved, we might need to reset  
  }  
  
  // 2. Injection Phase (Ray Tracing)  
  // We shoot rays for every pixel (or a subset)  
  // ...  
  
  // 3. Propagation Phase (Fluid Dynamics)  
  evolveSimulation();  
  
  // 4. Rendering Phase  
  // Draw the result to the Canvas  
  // ...  
  
  // 5. Repeat!  
  requestAnimationFrame(mainSimulationLoop);  
}
```

1.4 Conclusion

We have journeyed from the humble pixel to a complex, hybrid fluid-light transport simulation.

By treating light as a fluid, we create beautiful, organic lighting effects that respond to the environment in real-time. We optimised this by understanding computer memory (SoA) and employing simplified physics (Cellular Automata) rather than calculating every single photon perfectly.

Furthermore, our engine now monitors **Grid Sparsity**—the ratio of active light regions to empty space. This telemetry allows us to gauge efficiency and ensure we only process pixels that truly matter.

This is the power of the **Hybrid Fluid-Light Transport Engine**.